Lecture 16: Training Neural Networks

ECE 2410 – Introduction to Machine Learning

Farzad Farnoud

University of Virginia

Spring 2026

Today's Agenda

Where We Left Off

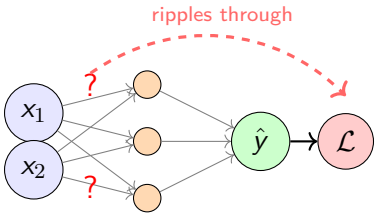
L15 — Three Ingredients for Training:

- ✅ **Training data** — examples $(\boldsymbol{x}_i, y_i)$
- ✅ **Loss function** — MSE or cross-entropy
- 🤔 **Optimization** — gradient descent (L13)…but how?

In linear regression, the gradient was easy — $\hat{y} = \langle \boldsymbol{w}, \boldsymbol{x} \rangle + b$ is one step from weights to prediction.

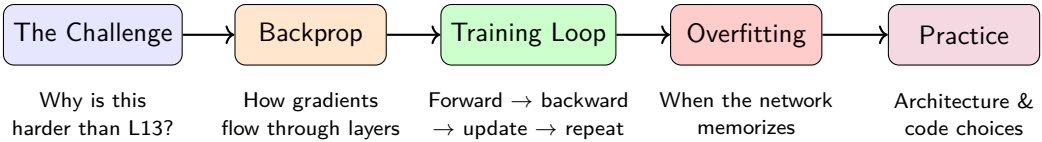
The New Challenge

In a neural network, weights are **buried inside layers**. Changing a hidden weight ripples through activation → next layer → $\cdots$ → loss.



Changing a hidden weight affects everything downstream.

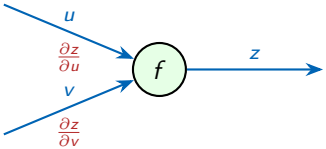
Today's Journey



By the end of today, you will be able to:

- Explain how gradients flow backward through a network
- Describe the full training loop (forward → loss → backward → update)
- Recognize overfitting and name two fixes
- Choose architecture and training settings for a new problem

Outline



How much does each input affect the output?

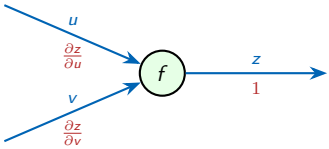
Impact Scores — Increasing a Function's Output

How to increase the output of a function $z = f(u, v)$ by changing u and v in the most efficient way possible:

- keep changes small
- maximize the increase in z

If we nudge the inputs by tiny amounts:

$$\Delta z \approx \frac{\partial z}{\partial u}\Delta u + \frac{\partial z}{\partial v}\Delta v$$

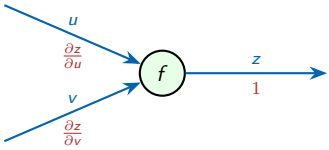


Blue (above): variable values.
Red (below): impact scores.

Impact Scores — Increasing a Function's Output

How to increase the output of a function $z = f(u, v)$ by changing u and v in the most efficient way possible:

- keep changes small
- maximize the increase in z



If we nudge the inputs by tiny amounts:

$$\Delta z \approx \frac{\partial z}{\partial u}\Delta u + \frac{\partial z}{\partial v}\Delta v$$

Blue (above): variable values.
Red (below): impact scores.

The impact of changing u is $\frac{\partial z}{\partial u}$ and the impact of changing v is $\frac{\partial z}{\partial v}$.
We call these values the **impact score** of each variable on the output.

Impact Scores — Example

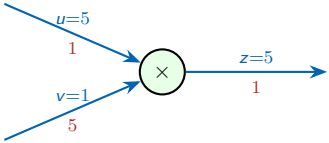
Example: $z = u \cdot v$ with u=5, v=1.

$$z = u \cdot v = 5 \times 1 = 5$$

$$\frac{\partial z}{\partial u} = v = 1, \qquad \frac{\partial z}{\partial v} = u = 5$$

$$\Delta z \approx 1 \cdot \Delta u + 5 \cdot \Delta v$$

The impact score for each input is the *other* input's value — this is a property of multiplication.



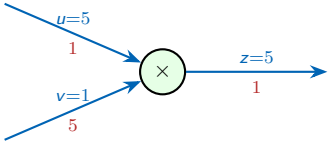
- Nudging v by η changes z by $5\,\eta$.
- Nudging u by η changes z by $1 \cdot \eta$.
- Nudging z by η changes z by $1 \cdot \eta$. (trivial)

How to Best Increase z?

From the previous slide: $\frac{\partial z}{\partial u} = 1, \ \frac{\partial z}{\partial v} = 5.$

$$\Delta z \ \approx \ 1 \cdot \Delta u \ + \ 5 \cdot \Delta v$$

Question: How to choose Δu and Δv to increase z as much as possible?



How to Best Increase z?

From the previous slide: $\frac{\partial z}{\partial u} = 1, \ \frac{\partial z}{\partial v} = 5.$

$$\Delta z \ \approx \ 1 \cdot \Delta u \ + \ 5 \cdot \Delta v$$

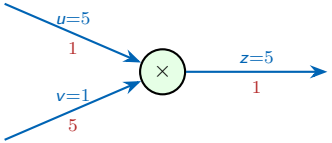
Question: How to choose Δu and Δv to increase z as much as possible?

Answer: Set each change **proportional to its impact score**:

$$\Delta u = 1 \cdot \eta, \qquad \Delta v = 5 \cdot \eta$$

$$\Delta z = 1 \cdot \eta + 5 \cdot 5\eta = \mathbf{26}\eta$$

To *decrease* z (e.g., a loss), flip the sign: $\Delta u = -\eta \cdot \frac{\partial z}{\partial u}$.

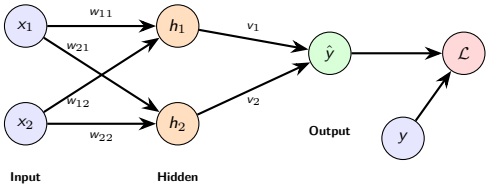


Key idea

Change each variable proportionally to how much it impacts the output.

The Network and the Goal

A $2 \to 2 \to 1$ network with loss:



The New Challenge

How do we compute $\frac{\partial \mathcal{L}}{\partial w_{ij}}$ when there are **many layers** between w_{ij} and $\mathcal{L}$?

We know how to decrease a function using impact scores:

$$\Delta\theta = -\eta \cdot \frac{\partial \mathcal{L}}{\partial \theta}$$

Here $\mathcal{L} = (\hat{y} - y)^2$ depends on **9 parameters** $(w_{11}, w_{12}, \ldots, v_1, v_2, d)$.

The data (x_1, x_2, y) is fixed — we can only adjust the parameters.

The answer is **backpropagation**: pass impact scores backward, one block at a time.

Outline

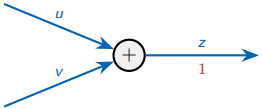
Simple operations
with simple rules

Every network is built from a handful of atomic operations.

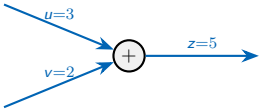
Building Block 1: Addition $z = u + v$

What is the impact of changing u and v on $z = u + v$?

General



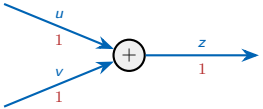
Example: $u{=}3,\ v{=}2$



Building Block 1: Addition $z = u + v$

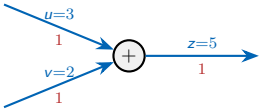
What is the impact of changing u and v on $z = u + v$?

General



$$\Delta z \approx 1 \cdot \Delta u + 1 \cdot \Delta v$$

Example: $u{=}3,\ v{=}2$



$$\Delta z \approx 1 \cdot \Delta u + 1 \cdot \Delta v$$

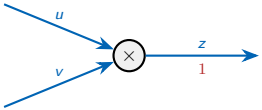
Backward messages for addition

We can view the impact scores as traveling backward through the network, one operation at a time. For addition, a 1 stays 1 as it propagates backward.

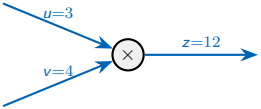
Building Block 2: Multiplication $z = u \cdot v$

Each input's impact score is the *other* input's value.

General



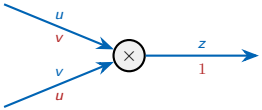
Example: $u{=}3,\ v{=}4$



Building Block 2: Multiplication $z = u \cdot v$

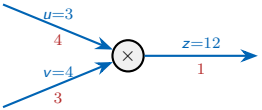
Each input's impact score is the *other* input's value.

General



$$\Delta z \approx v \cdot \Delta u + u \cdot \Delta v$$

Example: $u{=}3,\ v{=}4$



$$\Delta z \approx 4 \cdot \Delta u + 3 \cdot \Delta v$$

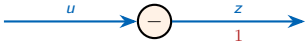
Backward messages for product

Each input amplifies the other input's impact. A 1 turns to the value of the other input as it propagates backward.

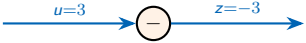
Building Block 3: Negation $z = -u$

Flips the sign of the impact score.

General



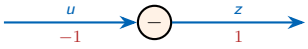
Example: $u{=}3$



Building Block 3: Negation $z = -u$

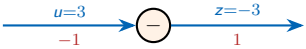
Flips the sign of the impact score.

General



$$\Delta z \approx (-1) \cdot \Delta u$$

Example: $u{=}3$



$$\Delta z \approx (-1) \cdot \Delta u$$

Backward messages for negation

A 1 becomes a -1 as it propagates backward.

Building Block 4: ReLU $z = \max(0, u)$

A gate: if $u > 0$, the signal passes through. If $u \leq 0$, the signal is blocked.

General

On: $u=3$

Off: $u=-2$

Building Block 4: ReLU $z = \max(0, u)$

A gate: if $u > 0$, the signal passes through. If $u \leq 0$, the signal is blocked.

General

$$\Delta z \approx \begin{cases} \Delta u & u>0 \\ 0 & u\leq 0 \end{cases}$$

On: $u=3$

$$\Delta z \approx 1 \cdot \Delta u$$

Off: $u=-2$

$$\Delta z \approx 0 \cdot \Delta u = 0$$

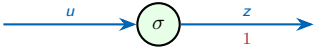
Backward messages for ReLU

When the gate is open ($u > 0$), a 1 passes through unchanged. When it is closed ($u \leq 0$), the message is blocked (multiplied by 0).

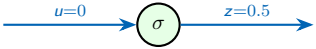
Building Block 5: Sigmoid $z = \sigma(u) = \dfrac{1}{1+e^{-u}}$

The impact score is $z(1-z)$ — it depends on the *output* value, not the input.

General



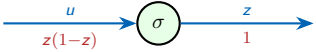
Example: $u=0$



Building Block 5: Sigmoid $z = \sigma(u) = \dfrac{1}{1+e^{-u}}$

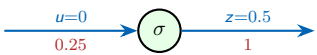
The impact score is $z(1-z)$ — it depends on the *output* value, not the input.

General



$$\Delta z \approx z(1-z) \cdot \Delta u$$

Example: $u=0$



$$\Delta z \approx 0.25 \cdot \Delta u$$

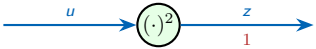
Backward messages for sigmoid

A 1 is scaled by $z(1-z)$. The impact is largest near $u=0$ and shrinks toward zero as $|u|$ grows.

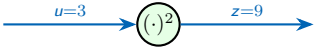
Building Block 6: Squaring $z = u^2$

The impact score is $2u$ — it depends on the input value.

General



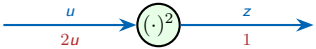
Example: $u{=}3$



Building Block 6: Squaring $z = u^2$

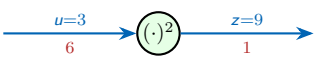
The impact score is $2u$ — it depends on the input value.

General



$$\Delta z \approx 2u \cdot \Delta u$$

Example: $u{=}3$



$$\Delta z \approx 6 \cdot \Delta u$$

Backward messages for squaring

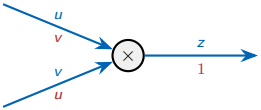
A 1 is scaled by $2u$. The larger $|u|$, the more impact a change in u has on the output.

We need negation and squaring for the loss: $\mathcal{L} = (\hat{y} - y)^2$.

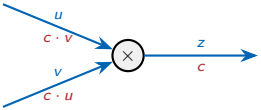
Scaling bacward messages

In every diagram so far, the impact score on the output edge was 1. But the incoming signal can be any value c:

Start signal $= 1$



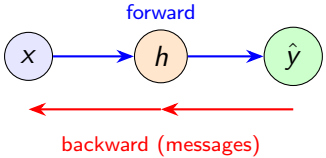
Start signal $= c$



Justification: $c\,\Delta z \approx c(v\,\Delta u + u\,\Delta v) = (cv)\,\Delta u + (cu)\,\Delta v$.

A **message** travels backward through each block. Each block multiplies the incoming message by its local impact score rule, and passes the result upstream.

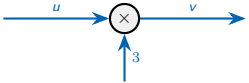
Outline



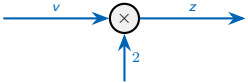
Chaining blocks together, then applying it to the full network.

Chaining Blocks — The Chain Rule

Block 1: $v = u \times 3$

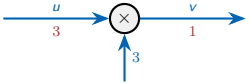


Block 2: $z = v \times 2$



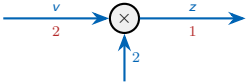
Chaining Blocks — The Chain Rule

Block 1: $v = u \times 3$



Changing u has a $3\times$ impact on v.

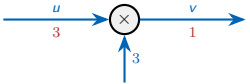
Block 2: $z = v \times 2$



Changing v has a $2\times$ impact on z.

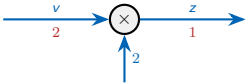
Chaining Blocks — The Chain Rule

Block 1: $v = u \times 3$



Changing u has a $3\times$ impact on v.

Block 2: $z = v \times 2$

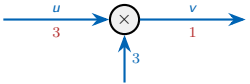


Changing v has a $2\times$ impact on z.

Chain them: What is the impact of changing u on z?

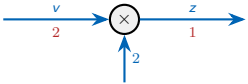
Chaining Blocks — The Chain Rule

Block 1: $v = u \times 3$



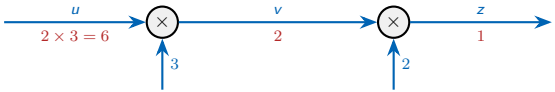
Changing u has a $3\times$ impact on v.

Block 2: $z = v \times 2$



Changing v has a $2\times$ impact on z.

Chain them: What is the impact of changing u on z?

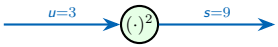


Verify: $z = 6u$, so $\frac{\partial z}{\partial u} = 6$. ✓ **This is the chain rule.**

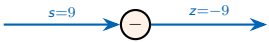
Chaining — A Non-Linear Example

Chain a squaring block and a negation block: first $s = u^2$, then $z = -s$. $(u{=}3)$

Block 1: $s = u^2$



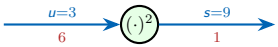
Block 2: $z = -s$



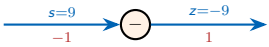
Chaining — A Non-Linear Example

Chain a squaring block and a negation block: first $s = u^2$, then $z = -s$. $(u{=}3)$

Block 1: $s = u^2$

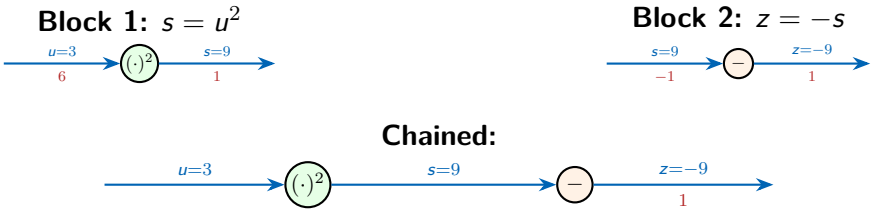


Block 2: $z = -s$



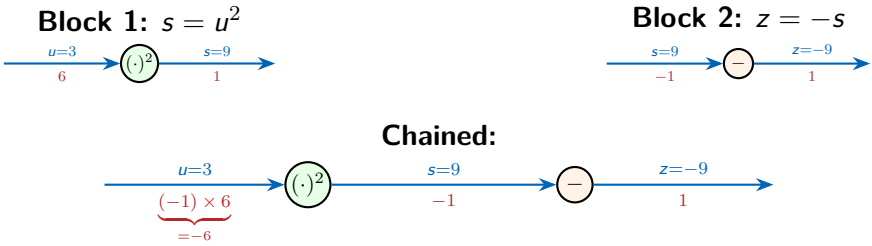
Chaining — A Non-Linear Example

Chain a squaring block and a negation block: first $s = u^2$, then $z = -s$. $(u{=}3)$



Chaining — A Non-Linear Example

Chain a squaring block and a negation block: first $s = u^2$, then $z = -s$. $(u{=}3)$



Verify: $z = -u^2$, so $\frac{\partial z}{\partial u} = -2u = -6$. ✓

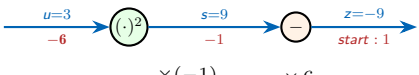
Chaining = Sending Backward Messages

Both examples follow the same pattern: a **message** starts at the output and each block multiplies it by its local rule.

Linear: $v = 3u, \quad z = 2v$

Message: $1 \xrightarrow{\times 2} 2 \xrightarrow{\times 3} 6$ ✓

Non-linear: $s = u^2, \quad z = -s$



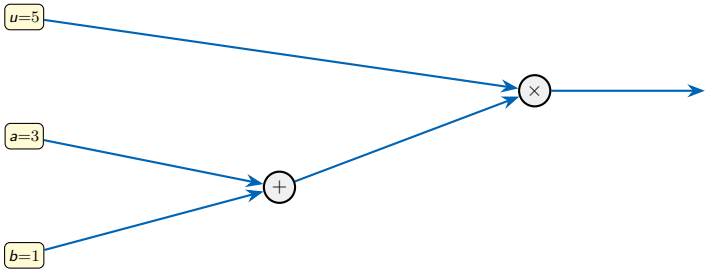
Message: $1 \xrightarrow{\times(-1)} -1 \xrightarrow{\times 6} -6$ ✓

The Backpropagation Rule

Start with message $= 1$ at the output. At each block, multiply the message by the block's local impact score rule, and pass it upstream.

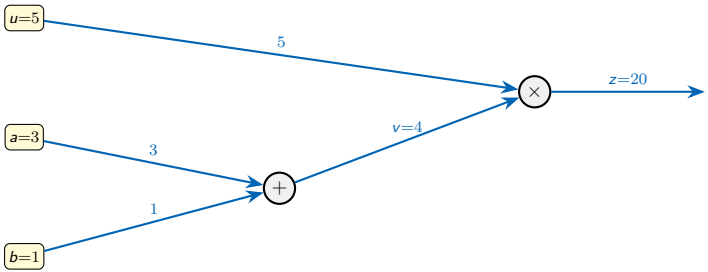
Chaining — Branching

First $v = a + b$, then $z = u \cdot v$. With u=5, a=3, b=1:



Chaining — Branching

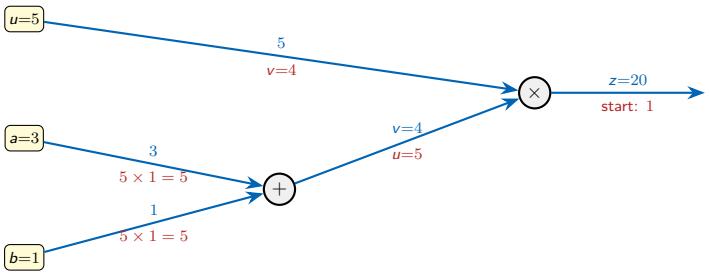
First $v = a + b$, then $z = u \cdot v$. With u=5, a=3, b=1:



Forward: $v = 3 + 1 = 4$, then $z = 5 \times 4 = 20$.

Chaining — Branching

First $v = a + b$, then $z = u \cdot v$. With u=5, a=3, b=1:



Forward: $v = 3 + 1 = 4$, then $z = 5 \times 4 = 20$.

Backward message: Start with 1 at z.

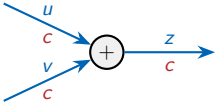
- At $\times$: message 1 splits — sends v=4 toward u, and u=5 toward v.
- At $+$: message 5 passes through unchanged to both a and b.

Verify: $z = u(a+b)$, so $\frac{\partial z}{\partial u} = 4$, $\frac{\partial z}{\partial a} = \frac{\partial z}{\partial b} = 5$. ✓

Backprop Blocks and Rules

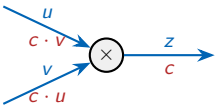
Any network can be broken down into a combination of the following blocks:

Addition $z = u + v$



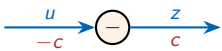
Pass through unchanged

Multiplication $z = u \cdot v$



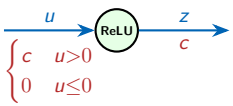
Scale by other input

Negation $z = -u$



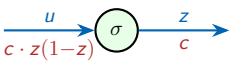
Flip the sign

ReLU $z = \max(0, u)$



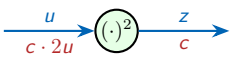
Gate: pass or block

Sigmoid $z = \sigma(u)$



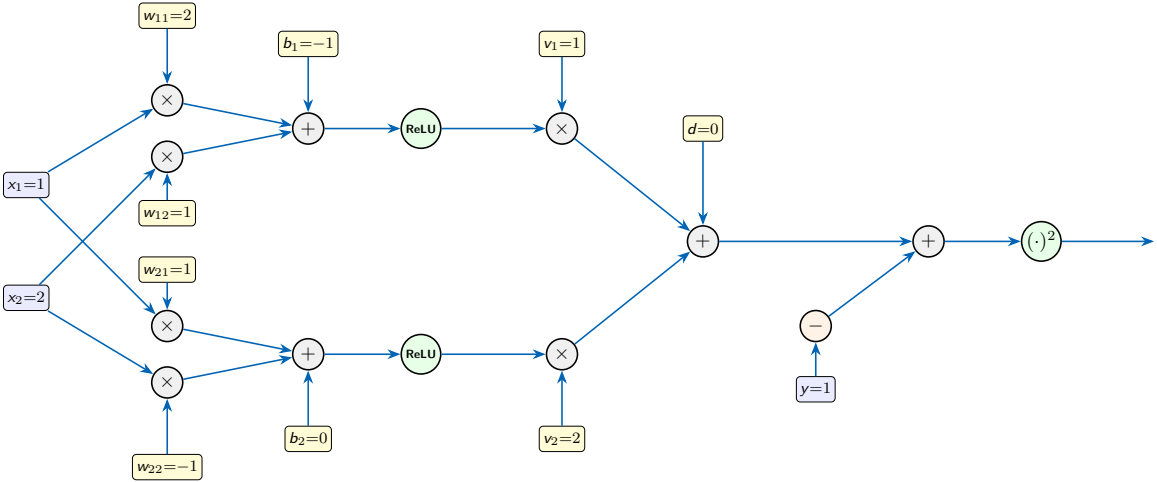
Scale by $z(1-z)$

Squaring $z = u^2$

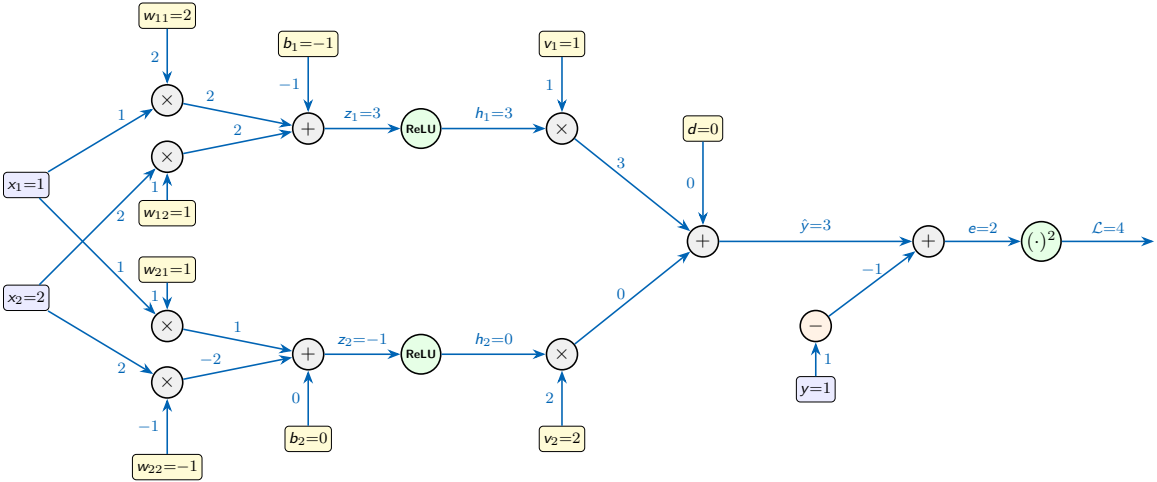


Scale by $2u$

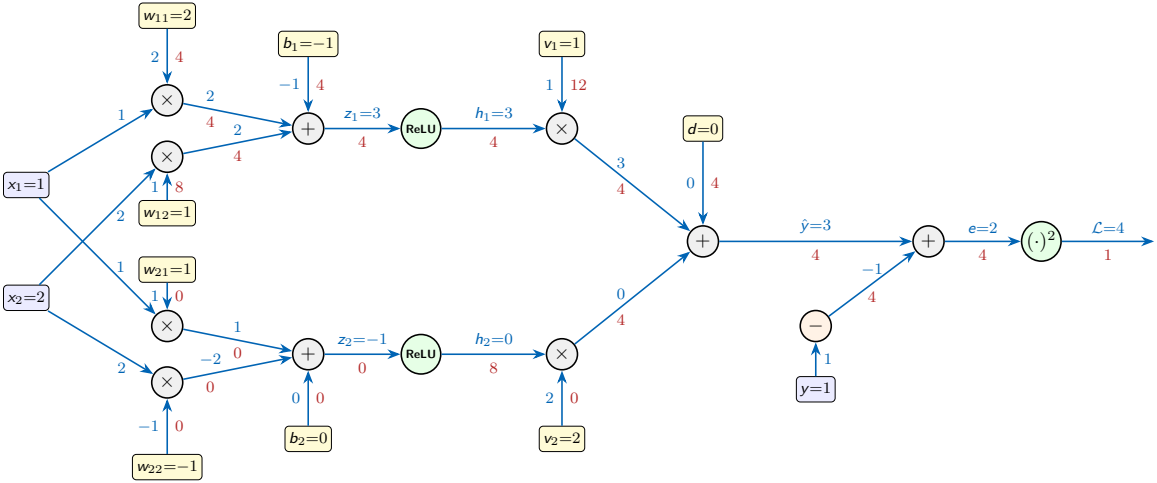
Backpropagation in the Full Network



Backpropagation in the Full Network



Backpropagation in the Full Network



The update rule

From the backward pass, every parameter got an **impact score** (how much it affects $\mathcal{L}$).

The update rule

From the backward pass, every parameter got an **impact score** (how much it affects $\mathcal{L}$).

Update Rule

Change each parameter w proportionally to its impact score:

$$\Delta w = -\eta \cdot \frac{\partial \mathcal{L}}{\partial w}.$$

E.g., $\Delta w_{11} = -\eta \cdot 4, \Delta b_2 = -\eta \cdot 0 = 0.$

The update rule

From the backward pass, every parameter got an **impact score** (how much it affects $\mathcal{L}$).

Update Rule

Change each parameter w proportionally to its impact score:

$$\Delta w = -\eta \cdot \frac{\partial \mathcal{L}}{\partial w}.$$

E.g., $\Delta w_{11} = -\eta \cdot 4, \Delta b_2 = -\eta \cdot 0 = 0.$

The total change in the loss is negative as long as η is small enough:

$$\Delta \mathcal{L} \approx \sum_w \frac{\partial \mathcal{L}}{\partial w} \Delta w = -\eta \sum_w \left(\frac{\partial \mathcal{L}}{\partial w} \right)^2.$$

Gradient and gradient descent

The **gradient** collects all impact scores into one vector:

$$\nabla \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial w_{11}}, \frac{\partial \mathcal{L}}{\partial w_{12}}, \frac{\partial \mathcal{L}}{\partial w_{21}}, \frac{\partial \mathcal{L}}{\partial w_{22}}, \frac{\partial \mathcal{L}}{\partial b_1}, \frac{\partial \mathcal{L}}{\partial b_2}, \frac{\partial \mathcal{L}}{\partial v_1}, \frac{\partial \mathcal{L}}{\partial v_2}, \frac{\partial \mathcal{L}}{\partial d} \right) = \left(4, 8, 0, 0, 4, 0, 12, 0, 4 \right)$$

The squared norm of the gradient is

$$\sum_w \left(\frac{\partial \mathcal{L}}{\partial w} \right)^2 = \|\nabla \mathcal{L}\|_2^2 = 4^2 + 8^2 + 0^2 + 0^2 + 4^2 + 0^2 + 12^2 + 0^2 + 4^2 = 240.$$

If $\eta = 0.01$, then $\Delta \mathcal{L} \approx -0.01 \times 240 = -2.4$.

Gradient and gradient descent

The **gradient** collects all impact scores into one vector:

$$\nabla \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial w_{11}}, \frac{\partial \mathcal{L}}{\partial w_{12}}, \frac{\partial \mathcal{L}}{\partial w_{21}}, \frac{\partial \mathcal{L}}{\partial w_{22}}, \frac{\partial \mathcal{L}}{\partial b_1}, \frac{\partial \mathcal{L}}{\partial b_2}, \frac{\partial \mathcal{L}}{\partial v_1}, \frac{\partial \mathcal{L}}{\partial v_2}, \frac{\partial \mathcal{L}}{\partial d} \right) = \left(4, 8, 0, 0, 4, 0, 12, 0, 4 \right)$$

The squared norm of the gradient is

$$\sum_{w} \left(\frac{\partial \mathcal{L}}{\partial w} \right)^2 = \|\nabla \mathcal{L}\|_2^2 = 4^2 + 8^2 + 0^2 + 0^2 + 4^2 + 0^2 + 12^2 + 0^2 + 4^2 = 240.$$

If $\eta = 0.01$, then $\Delta \mathcal{L} \approx -0.01 \times 240 = -2.4$.

Gradient descent update

The vector $\boldsymbol{w}$ is the collection of all parameters in the network: $\boldsymbol{w} \leftarrow \boldsymbol{w} - \eta \nabla \mathcal{L}$
The loss changes by $\Delta \mathcal{L} \approx -\eta \|\nabla \mathcal{L}\|^2$, assuming η is small enough.

From Impact Scores to Parameter Updates

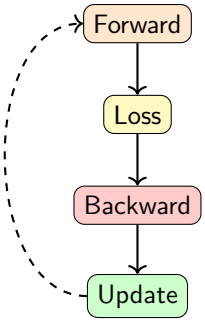
Multiple training examples:

With n examples (data points), the loss and the gradient are the average over the examples:

$$\mathcal{L}_{\text{total}} = \frac{1}{n}\sum_{i=1}^{n}\mathcal{L}^{(i)}, \qquad \nabla\mathcal{L}_{\text{total}} = \frac{1}{n}\sum_{i=1}^{n}\nabla\mathcal{L}^{(i)}$$

The impact score is simply the **average** across examples.
Also works when we have a batch of examples.

Outline

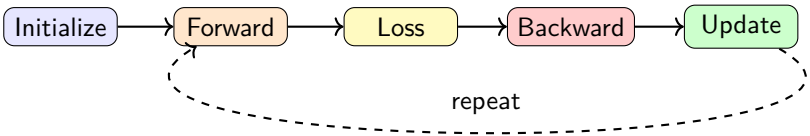


The forward → backward → update cycle.

Putting It Together — The Training Loop (Full Batch)

Training a Neural Network

1. **Initialize** all weights randomly
2. **Repeat** for many iterations:
   - **Forward pass:** compute prediction $\hat{y}^{(i)}$ and loss $\mathcal{L}^{(i)}$ for each example
   - **Backward pass:** run backprop on each example to get $\frac{\partial \mathcal{L}^{(i)}}{\partial w}$; average across examples
   - **Update:** $w \leftarrow w - \eta \cdot \frac{\partial \mathcal{L}_{\text{total}}}{\partial w}$ (gradient descent)
3. **Stop** when loss is small enough (or validation error increases)



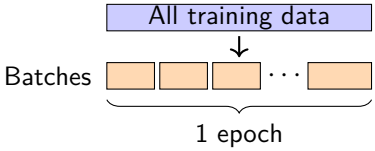
Epochs, Batches, and Iterations

For performance reasons, we don't use all the data at once. Mini-batch gradient descent uses a random **subset** each step.

Definitions

Batch: a subset of B training examples used for one gradient update.
Iteration: one gradient update (= one batch).
Epoch: one complete pass through all training data.

Example: 1000 training points, $B = 100$
$\rightarrow$ 10 batches per epoch $\rightarrow$ 10 iterations per epoch



Typical batch sizes: 16, 32, 64, 128

Training: 10–100+ epochs

Mini-Batch Gradient Descent

Pseudocode

1. **Initialize** all weights randomly
2. **For** each epoch $= 1, 2, \ldots$**:**
   1. Shuffle the training data
   2. Split into batches of size B
   3. **For** each batch**:**
      1. **Forward pass:** compute predictions and loss on this batch
      2. **Backward pass:** compute each data point's gradient, then average
      3. **Update:** $w \leftarrow w - \eta \cdot \text{gradient}$
3. **Stop** when validation loss stops improving

Key Insight

Weights are updated **once per batch**, not once per epoch. Smaller batches $\rightarrow$ more updates per epoch $\rightarrow$ noisier but faster learning.

Practice: Training Terminology

✏️ Your Turn

You have **6000 training images**, batch size $B = 200$, and you train for **5 epochs**.

How many **iterations** of gradient descent?

Practice: Training Terminology

✏️ Your Turn

You have **6000 training images**, batch size $B = 200$, and you train for **5 epochs**.

How many **iterations** of gradient descent?

Solution:

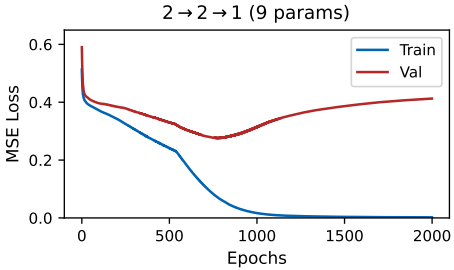
Batches per epoch: $6000/200 = 30$

Iterations: $30 \times 5 = \mathbf{150}$

That's 150 forward passes, 150 backward passes, and 150 weight updates.

Each of the 6000 images was used 5 times total (once per epoch).

Outline



When does the network memorize instead of learn?

Neural Networks Can Memorize

With enough parameters, a neural network can:

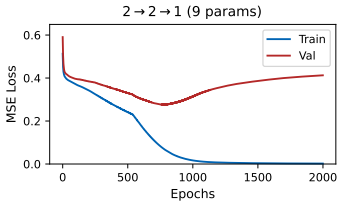
- Fit the training data **perfectly**
- Even memorize **random noise**
- Yet fail on new data

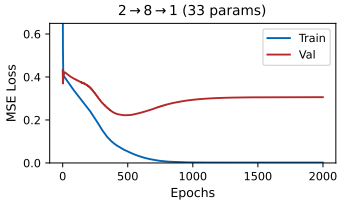
Classic symptom:

- Training loss keeps decreasing
- Validation loss starts **increasing**

Same bias-variance tradeoff from **L12**! More parameters = more capacity = more risk of overfitting.

Remember our XOR regression (8 points, 9 parameters)? Even this small network memorizes the noisy training data — train loss $\to 0$ while val loss rises.





Fix 1: Early Stopping

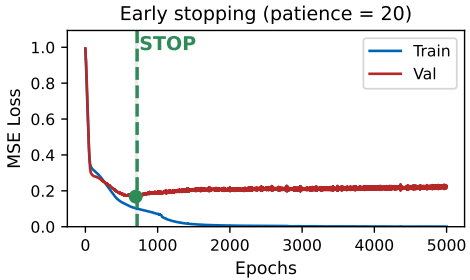
Early Stopping

Monitor **validation loss** during training. Stop when it starts to increase. Use the weights from the **best epoch**.

How it works:

1. Split data into train / validation / test
2. After each epoch, check validation loss
3. If no improvement for k epochs ("patience"), stop
4. Roll back to the best weights

Simple, effective, and free — just need a validation set.



Typical patience: $k = 5$ to 20 epochs.

Fix 2: Dropout

Dropout (Srivastava et al., 2014)

During training, randomly "turn off" each neuron with probability p (typically $p = 0.5$).

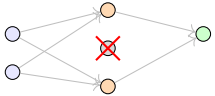
How it works:

- Each training step: randomly drop neurons (output $= 0$)
- Different neurons dropped each time
- At test time: use **all** neurons (scaled by $1 - p$)

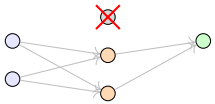
Intuition: no neuron can rely on another. Each must be useful on its own.

Like training many different sub-networks and averaging their predictions.

Training step 1



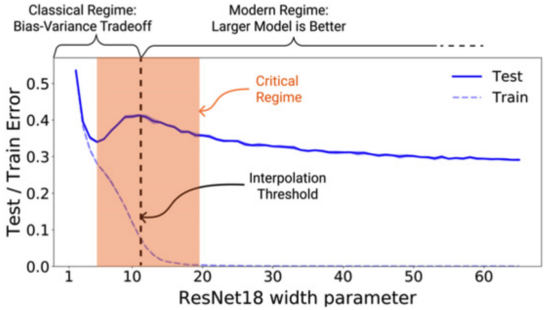
Training step 2



Different "sub-network" each step!

The Bigger Picture: Double Descent

We just said: more parameters → more overfitting. Full story is more interesting.



ResNet18 on CIFAR-10 (Nakkiran et al., *ICLR* 2020)

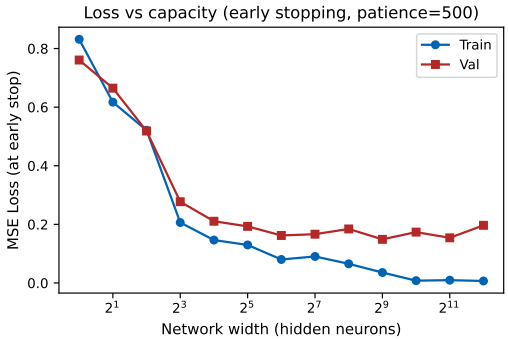
Three Regimes

1. **Classical:** the U-shape from L12
2. **Critical:** just enough parameters to memorize — worst test error
3. **Overparameterized:** keep adding — test error drops again!

Why? At the interpolation threshold, there's only *one* solution that fits (fragile). With many more parameters, GD finds a *smoother* one among many.

An Experiment

Our XOR experiment, with very large hidden layers (30 pts, early stopping)



Validation error decreases as we add neurons, then **flattens out** — it never goes back up.

So the right side of the U is missing. More parameters neither hurt nor help past a point.

Why? Early stopping may mask the effect — it prevents full memorization, so we never reach the regime where test error spikes. Also, our 30-point toy example may be too small to exhibit double descent.

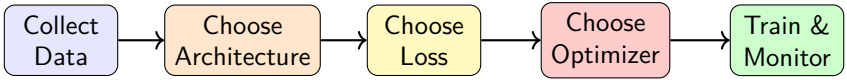
Takeaway (with a grain of salt)

With early stopping, even very large networks are resilient to overfitting. You can safely start big.

Outline

Putting the pieces together: architecture, loss, optimizer, and code.

The Practitioner's Pipeline



Hidden layers:

- Activation: **ReLU** (default choice)
- Start with **1 hidden layer**
- Start with **10–100 neurons**
- Add complexity only if underfitting

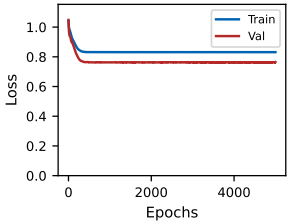
Rules of thumb: Start simple, add hidden neurons and layers if needed

Task determines the output:

	Regression	**Classification**
Output act.	none	sigmoid
Output range	$(-\infty, \infty)$	$(0, 1)$
Loss	MSE	cross-entropy

Reading Training Curves

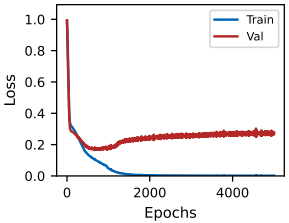
Underfitting



Both losses are **high**.

Fix: bigger network, more epochs, lower learning rate.

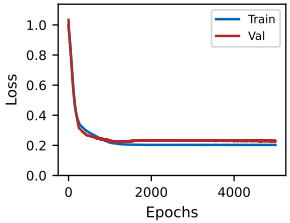
Overfitting



Train low, val **rising**. Big **gap**.

Fix: early stopping, dropout, more data.

Good Fit



Both low, small gap. Both **plateau**.

✅ Training is working!

Rule of Thumb

Train loss tells you about **capacity**. Val loss tells you about **generalization**. Watch the gap.

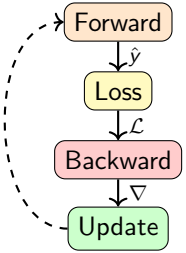
Summary: Training Neural Networks

Today we learned:

1. **The challenge:** gradients must trace through layers
2. **Backpropagation:** chain rule + local gradients $\rightarrow$ all gradients in one pass
3. **Training loop:** forward $\rightarrow$ loss $\rightarrow$ backward $\rightarrow$ update $\rightarrow$ repeat
4. **Mini-batches:** faster updates, noise can help
5. **Overfitting:** early stopping + dropout
6. **Practice:** architecture & code decisions map to concepts

Key insight: Backprop makes training feasible. Libraries handle it automatically.

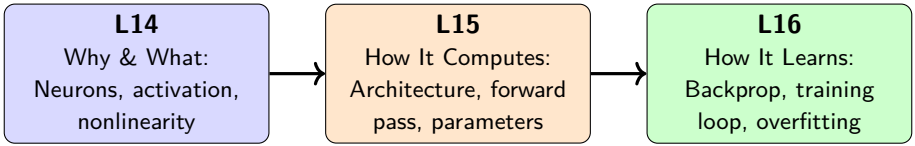
The Training Loop



Notebook

You'll train MLPs yourself and see these concepts in action!

The Full Neural Network Story



You now understand the full pipeline: **data** $\rightarrow$ **architecture** $\rightarrow$ **forward pass** $\rightarrow$ **loss** $\rightarrow$ **backprop** $\rightarrow$ **trained model**.

Next: L17 — Probability

A different branch of ML. Random variables, distributions, and how uncertainty drives predictions.